

Database Practicum

WEEK 4 ASSIGNMENT



Farrel Tsaqif Anindyo
24/536735/PA/22769

I. Student Activity

1. Create a Database

Create a database based on the given table. I decided to not include the 'no' column because there is already a unique column identifier for each record that is 'NIM' column. I set the 'NIM' column to be a primary key.

```
MariaDB [academic]> INSERT INTO student (NIM, Name, cityAddress, Age, IPK, Department) VALUES
-> (12345, 'Adi', 'Jakarta', 17, 2.5, 'Math'),
-> (12346, 'Ani', 'Yogyakarta', 20, 2.1, 'Math'),
-> (12347, 'Ari', 'Surabaya', 18, 2.5, 'Computer'),
-> (12348, 'Ali', 'Banjarasin', 20, 2.7, 'Computer'),
-> (12349, 'Abi', 'Medan', 17, 3.7, 'Computer'),
-> (12350, 'Budi', 'Jakarta', 19, 3.8, 'Computer'),
-> (12351, 'Boni', 'Yogyakarta', 20, 3.3, 'Computer'),
-> (12352, 'Bobi', 'Surabaya', 17, 2.7, 'Computer'),
-> (12353, 'Beni', 'Banjarasin', 16, 2.3, 'Computer'),
-> (12354, 'Cepi', 'Jakarta', 20, 2.2, NULL),
-> (12355, 'Comi', 'Yogyakarta', 18, 2.6, NULL),
-> (12356, 'Ceki', 'Surabaya', 21, 2.5, 'Math'),
-> (12357, 'Dodi', 'Jakarta', 20, 3.8, 'Math'),
-> (12358, 'Didi', 'Yogyakarta', 19, 3.2, 'Physics'),
-> (12359, 'Deri', 'Surabaya', 19, 3.3, 'Physics'),
-> (12360, 'Eli', 'Jakarta', 19, 2.9, 'Physics'),
-> (12361, 'Endah', 'Yogyakarta', 18, 2.8, 'Physics'),
-> (12362, 'Feni', 'Jakarta', 17, 3.5, 'Null'),
-> (12363, 'Farah', 'Yogyakarta', 18, 3.5, NULL),
-> (12364, 'Fandi', 'Surabaya', 19, 3.4, NULL);
Query OK, 20 rows affected (0.006 sec)
Records: 20 Duplicates: 0 Warnings: 0
```

```
MariaDB [academic]> select * from student;
+-----+-----+-----+-----+-----+-----+
| NIM | Name | cityAddress | Age | IPK | Department |
+-----+-----+-----+-----+-----+-----+
| 12345 | Adi | Jakarta | 17 | 2.50 | Math |
| 12346 | Ani | Yogyakarta | 20 | 2.10 | Math |
| 12347 | Ari | Surabaya | 18 | 2.50 | Computer |
| 12348 | Ali | Banjarasin | 20 | 2.70 | Computer |
| 12349 | Abi | Medan | 17 | 3.70 | Computer |
| 12350 | Budi | Jakarta | 19 | 3.80 | Computer |
| 12351 | Boni | Yogyakarta | 20 | 3.30 | Computer |
| 12352 | Bobi | Surabaya | 17 | 2.70 | Computer |
| 12353 | Beni | Banjarasin | 16 | 2.30 | Computer |
| 12354 | Cepi | Jakarta | 20 | 2.20 | NULL |
| 12355 | Comi | Yogyakarta | 18 | 2.60 | NULL |
| 12356 | Ceki | Surabaya | 21 | 2.50 | Math |
| 12357 | Dodi | Jakarta | 20 | 3.80 | Math |
| 12358 | Didi | Yogyakarta | 19 | 3.20 | Physics |
| 12359 | Deri | Surabaya | 19 | 3.30 | Physics |
| 12360 | Eli | Jakarta | 19 | 2.90 | Physics |
| 12361 | Endah | Yogyakarta | 18 | 2.80 | Physics |
| 12362 | Feni | Jakarta | 17 | 3.50 | Null |
| 12363 | Farah | Yogyakarta | 18 | 3.50 | NULL |
| 12364 | Fandi | Surabaya | 19 | 3.40 | NULL |
+-----+-----+-----+-----+-----+-----+
20 rows in set (0.001 sec)
```

2. Show a city address without duplicate

I use *distinct* in the *select* command to show a values without duplicates

```
MariaDB [academic]> select DISTINCT cityAddress  
-> from student;  
+-----+  
| cityAddress |  
+-----+  
| Jakarta    |  
| Yogyakarta |  
| Surabaya   |  
| Banjarmasin|  
| Medan      |  
+-----+  
5 rows in set (0.003 sec)
```

3. Display student names from the Computer Science Department

I use *where* with the condition

```
MariaDB [academic]> select Name  
-> from student  
-> where Department = 'Computer';  
+-----+  
| Name |  
+-----+  
| Ari  |  
| Ali  |  
| Abi  |  
| Budi |  
| Boni |  
| Bobi |  
| Beni |  
+-----+  
7 rows in set (0.001 sec)
```

4. Display student ID, name, age, and city address sorted by oldest student

I use *ORDER BY* with descending to order the output from the biggest value

```
MariaDB [academic]> select NIM, NAME, Age, cityAddress
-> from student
-> ORDER BY Age DESC;
```

NIM	NAME	Age	cityAddress
12356	Ceki	21	Surabaya
12346	Ani	20	Yogyakarta
12348	Ali	20	Banjarmasin
12351	Boni	20	Yogyakarta
12354	Cepi	20	Jakarta
12357	Dodi	20	Jakarta
12350	Budi	19	Jakarta
12358	Didi	19	Yogyakarta
12359	Deri	19	Surabaya
12360	Eli	19	Jakarta
12364	Fandi	19	Surabaya
12347	Ari	18	Surabaya
12355	Comi	18	Yogyakarta
12361	Endah	18	Yogyakarta
12363	Farah	18	Yogyakarta
12345	Adi	17	Jakarta
12349	Abi	17	Medan
12352	Bobo	17	Surabaya
12362	Feni	17	Jakarta
12353	Beni	16	Banjarmasin

20 rows in set (0.001 sec)

5. Display the names and ages of the three youngest students from Jakarta

I use *LIMIT*

```
MariaDB [academic]> select Name, Age
-> from student
-> ORDER BY Age
-> LIMIT 3;
```

Name	Age
Beni	16
Adi	17
Abi	17

3 rows in set (0.001 sec)

6. Display the names and GPAs of students from Jakarta with a GPA below 2.5

I use *AND* so it will return true/showing the output when it met the both conditions

```
MariaDB [academic]> select Name, IPK
-> from student
-> WHERE cityAddress = 'Jakarta' AND IPK < 2.5;
```

Name	IPK
Cepi	2.20

1 row in set (0.005 sec)

7. Display the names of all students from Yogyakarta or older than 20

I use *OR* so it will return true/showing the output when it met one of the conditions regardless the other value

```

MariaDB [academic]> select Name
-> from student
-> WHERE cityAddress = 'Yogyakarta' OR Age > 20;
+-----+
| Name |
+-----+
| Ani  |
| Boni |
| Comi |
| Ceki |
| Didi |
| Endah|
| Farah|
+-----+
7 rows in set (0.001 sec)

```

8. Display the names and addresses of students not from Jakarta or Surabaya

I use *NOT IN* so it will exclude the conditions

```

MariaDB [academic]> select Name, cityAddress
-> from student
-> WHERE cityAddress NOT IN ('Jakarta', 'Surabaya');
+-----+-----+
| Name | cityAddress |
+-----+-----+
| Ani  | Yogyakarta |
| Ali  | Banjarmasin|
| Abi  | Medan      |
| Boni | Yogyakarta |
| Beni | Banjarmasin|
| Comi | Yogyakarta |
| Didi | Yogyakarta |
| Endah| Yogyakarta |
| Farah| Yogyakarta |
+-----+-----+
9 rows in set (0.001 sec)

```

9. Display the names and ages of students with a GPA between 2.5 and 3.5

I use *BETWEEN* to get the output between two conditions

```

MariaDB [academic]> select Name, Age, IPK
-> from student
-> WHERE IPK BETWEEN 2.5 AND 3.5;
+-----+-----+-----+
| Name | Age | IPK |
+-----+-----+-----+
| Adi | 17 | 2.50 |
| Ari | 18 | 2.50 |
| Ali | 20 | 2.70 |
| Boni | 20 | 3.30 |
| Bobi | 17 | 2.70 |
| Comi | 18 | 2.60 |
| Ceki | 21 | 2.50 |
| Didi | 19 | 3.20 |
| Deri | 19 | 3.30 |
| Eli | 19 | 2.90 |
| Endah | 18 | 2.80 |
| Feni | 17 | 3.50 |
| Farah | 18 | 3.50 |
| Fandi | 19 | 3.40 |
+-----+-----+-----+
14 rows in set (0.001 sec)

```

10. Display the names of students with letters in their names

I use *WHERE* with the wildcard 'F%' to search for any strings that start with letter 'F' in the Name column

```

MariaDB [academic]> select Name
-> from student
-> WHERE Name like 'F%';
+-----+
| Name |
+-----+
| Feni |
| Farah |
| Fandi |
+-----+
3 rows in set (0.001 sec)

```

11. Display student IDs with null data

I use *IS NULL* to check whether there is a null/empty value

```
MariaDB [academic]> select NIM
-> from student
-> WHERE Department IS NULL;
+-----+
| NIM    |
+-----+
| 12354  |
| 12355  |
| 12363  |
| 12364  |
+-----+
4 rows in set (0.006 sec)
```

II. Questions

1. Mengapa kita perlu menggunakan DML?

Because it is the way for us to manipulate the data inside the database, like if you want to do a main data manipulation technique for example add a new value, edit the value, or delete the value, or you want to see what is the structure and the description of the data because without all of that the data become useless because you can't even see it or manipulate it.

2. Apa pentingnya data sorting?

This is important because sorting organizes raw data in a way that helps you read, interpret, and analyze it with more precision. Unsorted data is likely in no order at all and patterns or trends may be hard to spot. In other words, sorting takes raw data and turns it into organized information that enables clarity, accuracy and ultimately better decision-making. You can use the **`SORT BY`** command in mysql.

3. Apa perbedaan antara query dan filtering data?

A query is any request made to a database to retrieve, insert, update, or delete data. For example, `SELECT * FROM Customers;` is a query that retrieves all data from the Customers table.

Filtering data is a more specific action within a query. Filtering means applying conditions to the results so that only certain rows are returned. For example, `SELECT * FROM Customers WHERE City = 'Jakarta';` is still a query, but it includes a filter so that only customers from Jakarta are shown.

III. Exercise

1. Distinct

Display a list of countries where customers come from, without duplication

```
MariaDB [classicmodels]> select DISTINCT country  
-> from customers;
```

country
France
USA
Australia
Norway
Poland
Germany
Spain
Sweden
Denmark
Singapore
Portugal
Japan
Finland
UK
Ireland
Canada
Hong Kong
Italy
Switzerland
Netherlands
Belgium
New Zealand
South Africa
Austria
Philippines
Russia
Israel

2. WHERE + ORDER BY

Show 10 customers from the USA with a credit limit above 50,000, sorted by highest credit limit

```
MariaDB [classicmodels]> select customerName, country, creditLimit  
-> from customers  
-> WHERE country = 'USA' AND creditLimit > 50000  
-> ORDER BY creditLimit DESC;
```

customerName	country	creditLimit
Mini Gifts Distributors Ltd.	USA	210500.00
Muscle Machine Inc	USA	138500.00
Marta's Replicas Co.	USA	123700.00
Land of Toys Inc.	USA	114900.00
Online Diecast Creations Co.	USA	114200.00
Collectable Mini Designs Co.	USA	105000.00
Corporate Gift Ideas Co.	USA	105000.00
Mini Classics	USA	102700.00
Diecast Classics Inc.	USA	100600.00
Super Scale Inc.	USA	95400.00
Mini Creations Ltd.	USA	94500.00
Toys4GrownUps.com	USA	90700.00
FunGiftIdeas.com	USA	85800.00
Diecast Collectables	USA	85100.00
Technics Stores Inc.	USA	84600.00
Gift Depot Inc.	USA	84300.00
Classic Gift Ideas, Inc	USA	81100.00
The Sharp Gifts Warehouse	USA	77600.00
Vitachrome Inc.	USA	76400.00
Motor Mint Distributors Inc.	USA	72600.00
Signal Gift Stores	USA	71800.00
Collectables For Less Inc.	USA	70700.00
Online Mini Collectables	USA	68700.00
Classic Legends Inc.	USA	67500.00
Mini Wheels Co.	USA	64600.00
Signal Collectibles Ltd.	USA	60300.00
Men 'R' US Retailers, Ltd.	USA	57700.00
West Coast Collectables Co.	USA	55400.00

3. LIMIT

Take the 5 products with the highest selling price (MSRP)

```

MariaDB [classicmodels]> select productName, MSRP
  -> from products
  -> ORDER BY MSRP DESC
  -> LIMIT 5;
+-----+-----+
| productName | MSRP |
+-----+-----+
| 1952 Alpine Renault 1300 | 214.30 |
| 2001 Ferrari Enzo | 207.80 |
| 1968 Ford Mustang | 194.57 |
| 2003 Harley-Davidson Eagle Drag Bike | 193.66 |
| 1969 Ford Falcon | 173.02 |
+-----+-----+
5 rows in set (0.001 sec)

```

4. AND / OR

Display customer data from Paris or Madrid, with a credit limit of over 80,000.

```

MariaDB [classicmodels]> select customerName, country, creditLimit
  -> from customers
  -> WHERE (country = 'Paris' OR country = 'Madrid') AND creditLimit > 80000;
Empty set (0.007 sec)

```

5. IN

Display a list of customers from the following countries: USA, France, Japan.

```

MariaDB [classicmodels]> select customerName, country
-> from customers
-> WHERE country IN ('USA', 'France', 'Japan');

```

customerName	country
Atelier graphique	France
Signal Gift Stores	USA
La Rochelle Gifts	France
Mini Gifts Distributors Ltd.	USA
Mini Wheels Co.	USA
Land of Toys Inc.	USA
Saveley & Henriot, Co.	France
Muscle Machine Inc	USA
Diecast Classics Inc.	USA
Technics Stores Inc.	USA
American Souvenirs Inc	USA
Daedalus Designs Imports	France
La Corne D'abondance, Co.	France
Cambridge Collectables Co.	USA
Gift Depot Inc.	USA
Osaka Souvenirs Co.	Japan
Vitachrome Inc.	USA
Auto-Moto Classics Inc.	USA
Online Mini Collectables	USA
Toys4GrownUps.com	USA
Mini Caravy	France
Boards & Toys Co.	USA
Collectable Mini Designs Co.	USA
Alpha Cognac	France
Lyon Souvenirs	France
Auto Associates & Cie.	France
Marta's Replicas Co.	USA
Mini Classics	USA
Mini Creations Ltd.	USA

6. BETWEEN

Display products with a buyPrice between 50 and 100.

```

MariaDB [classicmodels]> select productName, buyPrice
-> from products
-> WHERE buyPrice BETWEEN 50 AND 100;

```

productName	buyPrice
1952 Alpine Renault 1300	98.58
1996 Moto Guzzi 1100i	68.99
2003 Harley-Davidson Eagle Drag Bike	91.02
1972 Alfa Romeo GTA	85.68
1968 Ford Mustang	95.34
2001 Ferrari Enzo	95.59
1958 Setra Bus	77.90
2002 Suzuki XRE0	66.27
1969 Corvair Monza	89.14
1968 Dodge Charger	75.16
1969 Ford Falcon	83.05
1957 Chevy Pickup	55.70
1969 Dodge Charger	58.73
1940 Ford Pickup Truck	58.33
1993 Mazda RX-7	83.51
1937 Lincoln Berline	60.62
1965 Aston Martin DB5	65.96
1980s Black Hawk Helicopter	77.27
1917 Grand Touring Sedan	86.70
1948 Porsche 356-A Roadster	53.90
1995 Honda Civic	93.89
1964 Mercedes Tour Bus	74.86
1932 Model A Ford J-Coupe	58.48
1928 Mercedes-Benz SSK	72.56
1999 Indy 500 Monte Carlo SS	56.76
1913 Ford Model T Speedster	60.78
1999 Yamaha Speed Boat	51.61
18th Century Vintage Horse Carriage	60.74
1903 Ford Model A	68.30
1992 Ferrari 360 Spider red	77.90

7. LIKE

Show all products whose product name contains the word "Ford".

```

MariaDB [classicmodels]> select productName
-> from products
-> WHERE productName LIKE '%Ford%';
+-----+
| productName |
+-----+
| 1968 Ford Mustang |
| 1969 Ford Falcon |
| 1940 Ford Pickup Truck |
| 1911 Ford Town Car |
| 1932 Model A Ford J-Coupe |
| 1926 Ford Fire Engine |
| 1913 Ford Model T Speedster |
| 1934 Ford V8 Coupe |
| 1903 Ford Model A |
| 1976 Ford Gran Torino |
| 1940s Ford truck |
| 1957 Ford Thunderbird |
| 1912 Ford Model T Delivery Wagon |
| 1940 Ford Delivery Sedan |
| 1928 Ford Phaeton Deluxe |
+-----+
15 rows in set (0.003 sec)

```

8. IS NULL

Display all orders that do not yet have a shipped date.

```
MariaDB [classicmodels]> select orderNumber  
-> from orders  
-> WHERE shippedDate IS NULL;
```

orderNumber
10167
10248
10260
10262
10334
10401
10407
10414
10420
10421
10422
10423
10424
10425

```
14 rows in set (0.004 sec)
```